

Name: _____





BOOTSTRAP

Equity • Scale • Rigor

Brought to you by the Bootstrap team:

- Emmanuel Schanzer
- Kathi Fiser
- Shriram Krishnamurthi
- Dorai Sitaram
- Joe Politz
- Ben Lerner
- Nancy Pfenning
- Flannery Denny
- Rachel Tabak

Sorting and Trimming Tables

For this page you'll need the [Spell Checker Starter File](#) open on your computer, with `results = alt-words("plation", WORDS-L, 4)` defined in the **Definitions Area**. Make sure that after clicking "Run", you can evaluate `results` in the Interactions Area and see your table!

Sorting Results

1-typo errors are more common than 2- or 3-typo errors, so we'd like to **sort the suggestions**, putting the more likely suggestions at the top.

1) Test out these two expressions and record what you learn below:

- `sort(results, "edit-distance", false)` _____
- `sort(results, "word", true)` _____

2) You've seen data types like Numbers, Strings, and Booleans. What data type does the `sort` function produce? _____

3) The `sort` function takes in three pieces of information (called arguments). Record what you know about them below.

	Data Type	Information it Contains
Argument 1	Table	The table being sorted
Argument 2	String	
Argument 3		

There are many other functions available to us in Pyret. We can describe them using **contracts**. The Contract for `sort` is:

```
# sort :: Table, String, Boolean -> Table
```

- Each Contract begins with the function name: *in this case* `sort` _____
- Lists the data types required to satisfy its **Domain**: *in this case* `Table, String, Boolean` _____
- And then declares the data type of the **Range** it will return: *in this case* `Table` _____

Contracts can also be written with more detail, by annotating the Domain with *variable names*:

```
# sort :: ( Table , String , Boolean ) -> Table
           t      column ascending
```

4) What code will sort `results` in alphabetical order, from A-Z? _____

Trimming Results

When we make a typo, our cell phones only show us 3-5 suggestions. We'd like to trim the table, showing only the first handful of suggestions.

5) Type `first-n-rows(results, 10)`. What happens? _____

6) If we wanted a table of the first 5 rows of the `results` table, what code would we write? _____

7) How many arguments are the **Domain** of `first-n-rows`? _____ What is the **Range**? _____

8) Using the contract for `sort` as an example, can you write the contract for `first-n-rows`?

_____ :: _____ -> _____
function name Domain Range

9) Pyret also has a function called `last-n-rows`. **Make a prediction:** What do you think its contract is?

_____ :: _____ -> _____
function name Domain Range

10) Test it out in Pyret! Was your prediction right?

User Interface for our Spelling Checker

For this page you'll need the [Spell Checker Starter File](#) open on your computer, with `results = alt-words("plation", WORDS-L, 4)` defined in the **Definitions Area**. Make sure that after clicking "Run", you can evaluate `results` in the Interactions Area and see your table!

1) What are the first two words in the `results` table? _____

2) What code will extract the `"word"` column from the *first row* in the table? _____

Here are some contracts (and examples!) for a few new functions:

Name	Domain	Range
<code>string-length</code>	<code>:: String</code>	<code>-> Number</code>
<code>text</code>	<code>::</code>	<code>-></code>

3) `string-length("hello")` evaluates to _____. What does `string-length` do? It counts the number of characters in a string

4) **Predict before you type:** For each expression below, what do you *think* Pyret will produce?

- `string-length(row-n(results, 1)["word"])?` _____ *Type it in and try it out!*
- `string-length(row-n(results, 2)["word"])?` _____ *Type it in and try it out!*

5) Evaluate `text("AI!", 50, "blue")`. What did you get back? _____

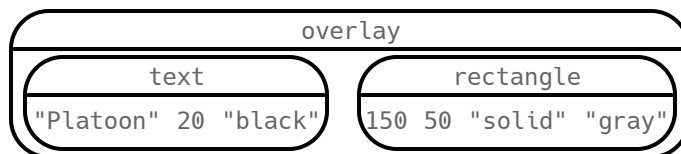
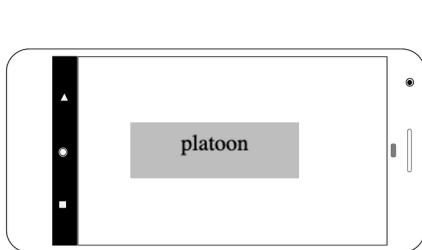
6) What code will draw the word in the *second row* of the `results` table in big, blue letters?

`text(row-n(results, 2)["word"], 50, "blue")`

Most of the functions below are all **missing their contracts!** Can you experiment in Pyret and fill in the missing parts?

Name	Domain	Range
<code>triangle</code>	<code>:: Number, String, String</code>	<code>-> Image</code>
<code>star</code>	<code>::</code>	<code>-></code>
<code>circle</code>	<code>::</code>	<code>-></code>
<code>rectangle</code>	<code>::</code>	<code>-></code>
<code>ellipse</code>	<code>::</code>	<code>-></code>
<code>regular-polygon</code>	<code>::</code>	<code>-></code>
<code>overlay</code>	<code>::</code>	<code>-></code>
<code>rotate</code>	<code>::</code>	<code>-></code>

7) The Circle of Evaluation below overlays the word "Platoon" above a solid, gray rectangle. Convert it to code and enter it into Pyret.



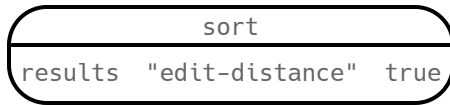
★ **Be Creative:** Use the image contracts above to add color, multiple shapes (maybe even multiple suggestions?) to your User Interface!

Composing with Circles of Evaluation

Notice and Wonder

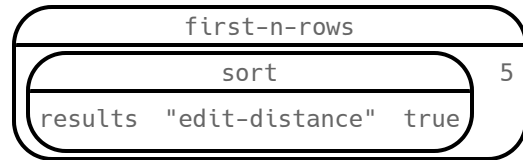
Suppose we want to see the top 5 results of `results`, sorted by increasing edit-distance. Let's use Circles of Evaluation to look at the structure:

We can start by generating the sorted table.



→

And then use `first-n-rows` to grab the top 5



```
sort(results, "edit-distance", true)
```

```
first-n-rows(sort(results, "edit-distance", true), 5)
```

1) What do you Notice? _____

2) What do you Wonder? _____

Mixing it Up

Suppose you wanted the computer to take the first 200 rows of `results`, and sort them in decreasing order of edit-distance...

Take the first 200 rows

Sort them

3) Draw the circle of evaluation:

4) Draw the circle of evaluation:

5) Convert the Circle of Evaluation to code:

6) Convert the Circle of Evaluation to code:

Data Visualization for Tables

Frequency Tables, Bar Charts and Pie Charts

For this page you'll need the [Spell Checker Starter File](#) open on your computer, with `results = alt-words("plation", WORDS-L, 4)` defined in the **Definitions Area**. Make sure that after clicking "Run", you can evaluate `results` in the Interactions Area and see your table!

- 1) Scroll through all the suggested words in `results`. Which `edit-distance` is the most common? _____ Least common? _____
- 2) Evaluate `count(results, "edit-distance")`. What did you get back? _____
- 3) Write the contract for Pyret's `count` function below:

_____ :: _____ -> _____
function name Domain Range

The contract for Pyret's `bar-chart` function is `# bar-chart :: Table, String -> Image`

- 4) Make a bar chart of your `results` table, using "edit-distance" for your column. What did you get back? _____
- 5) Move your cursor over one of the bars. What information does this tell you? _____
- 6) How is this related to the table you got from `count`? _____
- 7) Try making a bar chart using the "word" column. Is this visualization *useful*? Why or why not? _____

The contract for Pyret's `pie-chart` function is `# pie-chart :: Table, String -> Image`

- 8) Make a pie chart of your `results` table, using "edit-distance" for the column.

Real Spell Checkers Return Words, not Tables!

Let's define a row. At the bottom of the Definitions Area, `type row2 = row-n(results, 1)`. Click "Run".

- 9) Evaluate `row2` in the Interactions Area. What do you get back? _____
- 10) In your own words, explain what `row-n` does. _____
- 11) Complete the contract for `row-n` below. We've provided the Range for you.

_____ :: _____ -> _____
function name Domain Row Range

- 12) What does `row2["word"]` evaluate to? _____ `row2["edit-distance"]`? _____
- 13) Evaluate `row-n(results, 1)["word"]`. What did you get back? _____

Code like this (`row2["word"]` or `row2["edit-distance"]`) is called a **row accessor**.

- 14) In your own words, what do row accessors do? _____
- 15) What code will produce the *first* suggested word from our table? _____
- 16) How can we get the `edit-distance` of the *fifth* suggested word from our table? _____

These materials were developed partly through support of the National Science Foundation (awards 1042210, 1535276, 1648684, and 1738598) and are licensed under a Creative Commons 4.0 Unported License. Based on a work at www.BootstrapWorld.org. Permissions beyond the scope of this license may be available by contacting contact@BootstrapWorld.org.